MotoTrack — Rencana Akhir & Laporan Proyek

Pertanyaan yang dijawab: dari video motovlog ego-POV, apakah motor bisa nyalip ke depan atau tidak?

Status: selesai dan ter-deploy. 115 tes lulus. 10 commit di `github.com/rynnfrds/MotoTrack`.

Tanggal: 6 November 2026

---

1. Ringkasan eksekutif

MotoTrack menerima klip motovlog, mendeteksi dan melacak pengguna jalan, memproyeksikan koridor ego, mengukur ruang lateral bebas di dalamnya, lalu memberi vonis per momen: `SAFE` / `TIGHT` / `BLOCKED` / `UNKNOWN`.

Sistem berjalan penuh di CPU tanpa GPU. Waktu proses ~94 ms per frame yang dianalisis, sekitar 8,5 fps. Klip 15 detik selesai dalam ~17 detik.

Yang penting dipahami: **deteksi objek bukan kontribusi proyek ini** — itu sudah diselesaikan orang lain dan kita pakai bobot siap pakai. Kontribusi aslinya ada di lapisan keputusan: geometri koridor, normalisasi gap yang bebas kalibrasi, dan ambang yang diturunkan dari data berlabel, bukan ditebak.

---

2. Keputusan model (hasil riset)

Sebelum membangun, saya riset dulu apakah sudah ada model siap pakai. Hasilnya:

Kebutuhan	Yang tersedia	Keputusan
Deteksi kendaraan	YOLO26n (COCO), rilis v8.4.0	Dipakai, tanpa training
Deteksi lalu lintas campuran	fine-tune `visaitech/vehicle-mixed-traffic-yolo26n`	Dipakai sebagai alternatif
Estimasi kedalaman	`yolo26n-depth.pt`	Diunduh, tidak dipakai di jalur utama
Model "bisa nyalip?"	**tidak ada**	**dibuat sendiri: logika aturan + geometri**
Dataset POV Indonesia	**tidak ada**	pakai footage sendiri + dataset Asia Selatan

Catatan penting: **YOLO27 belum rilis.** Dokumentasi resmi menyatakan model belum tersedia dan belum ada tanggal peluncuran. Jadi YOLO26n adalah pilihan terbaru yang nyata, dan di CPU ia ~43% lebih cepat dari YOLO11n.

Konsekuensi yang jujur: tidak ada satu pun bobot yang dilatih di proyek ini. Semua checkpoint pihak ketiga, berlisensi AGPL-3.0.

---

3. Arsitektur

```
video  ->  deteksi + tracking (YOLO26n)
       ->  estimasi jarak (pinhole, asumsi tinggi objek + HFOV 78 derajat)
       ->  koridor ego (trapesium, koordinat relatif)
       ->  gap lateral bebas / lebar median kendaraan SEBANDING
       ->  vonis per frame (aturan + override)
       ->  stabilisasi temporal (median 2 s + histeresis 0,5 s)
       ->  annotated.mp4 + events.json + summary.json + timeline.png
```

Dua keputusan desain yang menentukan

a. Koridor dalam koordinat relatif. Trapesium didefinisikan sebagai fraksi lebar dan tinggi frame, jadi video 16:9 dan reels 9:16 memakai satu jalur kode yang sama.

b. Gap dinormalisasi lebar kendaraan, bukan piksel.

```
gap_ratio = gap_bebas_px / lebar_median_kendaraan_px   (pada band kedalaman yang
sama)
```

Karena pembagi dan pembilang diskalakan oleh perspektif yang sama, rasionya **invarian skala**. Artinya: tidak perlu kalibrasi kamera sama sekali. Ini yang membuat proyek bisa jalan di video apa pun tanpa setup.

---

4. Ambang: diturunkan, bukan dikarang

Ambang awal (1,6 / 1,2) adalah tebakan saya. Itu tidak layak dikirim, jadi saya buat `scripts/calibrate.py` yang menyapu 780 pasangan ambang terhadap gold set berlabel manual.

Aturan pemilihannya sengaja asimetris: pilih pasangan dengan kesepakatan tertinggi **dengan syarat tidak pernah menyebut SAFE pada frame yang manusia sebut BLOCKED**.

Ambang	Akurasi 3 kelas	Akurasi biner	SAFE padahal BLOCKED
Tebakan (1,6 / 1,2)	20,7%	62,1%	0
Hasil fit (1,8 / 0,4)	**41,4%**	**69,0%**	**0**

Alasannya: SAFE palsu mengundang pengendara mengambil manuver yang tidak bisa dibatalkan. BLOCKED palsu hanya membuang waktu. Semua keputusan smoothing condong ke arah aman.

---

5. Hasil pengukuran

Kondisi: 1 klip (`clip_001.mp4`), 640x480, 20 fps, 14,65 detik, 293 frame. Gold set: 29 frame berlabel manual oleh satu orang. CPU saja.

Perbandingan detektor

Detektor	Akurasi biner	Akurasi 3 kelas	Dinilai	UNKNOWN
`coco`	41,7%	33,3%	12	17
`mixed`	**69,0%**	**41,4%**	29	0

Baris `coco` menyesatkan dengan cara yang berguna: ia abstain di 17 dari 29 frame. Model mixed menjawab semua frame dan lebih sering benar.

Yang TIDAK berhasil

Akurasi 3 kelas 41,4% **belum layak pakai.** Recall TIGHT hanya 8,7%, BLOCKED 0%. Sistem condong bilang BLOCKED — aman, tapi tidak berguna.

Angka ini ditulis apa adanya di `docs/METRICS.md`. Dua hal yang harus disebut bersamaan: ambang di-fit **pada 29 frame yang sama** yang dipakai menilai, jadi 41,4% itu **optimistis**, bukan hasil validasi.

---

6. Bug yang ditemukan dan diperbaiki

Ini bagian yang paling menunjukkan cara kerja sebenarnya.

#	Bug	Dampak terukur	Perbaikan
1	Masker bodywork motor sendiri pakai ambang bawah frame saja	Menelan mobil 1,5 m di depan — objek paling penting	Butuh dua syarat: menempel bawah **dan** >=75% lebar frame
2	`gap_ratio` dinormalisasi lebar median **semua** deteksi	Satu box jauh 30 px memperkecil pembagi, rasio menggelembung	Pembagi diambil dari band kedalaman yang sama
3	`UNKNOWN` diikutkan dalam median	76,9% frame jadi UNKNOWN	UNKNOWN dikecualikan (artinya "tidak berpendapat")
4	Frame yang dilewati stride ditulis mentah	5 dari 6 frame output tanpa overlay di 60 fps → kedip	Digambar dengan state terakhir
5	Menahan state terakhir membekukan overlay	67% pasangan frame identik → box melompat	Buffer + interpolasi antar dua state → 10%
6	Track hilang-timbul 1-2 sampel	56 event hilang-muncul	EMA sudut box + coasting 0,4 s → 10 event
7	`/api/health` melaporkan port default	Info palsu saat desktop pindah port	Melaporkan port yang benar-benar terikat

Bug 4–6 semuanya **soal tampilan, bukan akurasi** — dan itu dinyatakan eksplisit di dokumentasi. Mudah sekali membuat demo terlihat lebih tenang lalu menyimpulkan modelnya jadi lebih pintar. Bukan.

Setiap bug punya tes regresi. Saya buktikan tes itu benar-benar menangkap: dengan bug dikembalikan, tes gagal (mis. `19.4 > 10.0`, rasio frame bergerak jatuh ke 15%).

---

7. Yang dibangun

Komponen	Isi
Pipeline	deteksi, tracking, geometri, vonis, stabilisasi, output
CLI	`analyze`, `preview-corridor`, `eval`, `compare-detectors`, `web`, `desktop`
Web app	upload, progress nyata, halaman hasil, timeline, export JSON
Aplikasi Windows	`Start MotoTrack.bat` → jendela native, tanpa browser
Evaluasi	gold set, matriks konfusi, kalibrasi ambang
Notebook Colab	fine-tune opsional, data-prep terverifikasi
Dokumentasi	README, model card, metrics, demo runbook
Tes	115 tes

Port

Default **8770**. Port 8000 sengaja dihindari karena dipakai aplikasi lain di mesin ini. Bisa dioverride via `--port` atau `MOTOTRACK_PORT`. CLI memeriksa port dulu dan menyarankan alternatif alih-alih gagal separuh jalan. Aplikasi desktop pindah port otomatis. Tidak ada proses lain yang pernah dimatikan.

---

8. Batasan yang harus dibaca sebelum dipakai

Ini bukan sistem keselamatan. Jangan dipakai untuk berkendara sungguhan.

Situasi	Perilaku
Malam, hujan, terowongan, silau	Deteksi kolaps → `UNKNOWN`, bukan menebak
Motor miring saat menikung	Koridor tetap di frame, jalan keluar dari koridor
Lalu lintas lawan arah	Tidak dimodelkan
Kendaraan mau pindah lajur	Tidak dimodelkan, tidak ada prediksi gerak
Bus dianggap mobil	Kelas `4-wheeler` menyatukan sedan–bus, tinggi asumsi salah
Pejalan kaki	**Data latih hanya 30 instance vs 731 motor** — paling lemah di objek paling berbahaya

Jarak bersifat estimasi monokular dari asumsi tinggi objek dan HFOV 78 derajat — bukan pengukuran sensor. Dilaporkan sebagai band (dekat <3 m, sedang <8 m, jauh), dengan peringatan di HUD.

---

9. Kalau dilanjutkan, urutan prioritasnya

Prioritas	Langkah	Alasan
1	Tambah klip sampai gold set ~150 frame	29 frame terlalu sedikit; 41,4% itu optimistis
2	Pisahkan data fit dan data uji	Sekarang ambang di-fit pada set yang sama yang dinilai
3	Tangani lean/miring	Penyebab gagal paling nyata di footage nyata
4	Fine-tune untuk pejalan kaki	30 instance tidak cukup untuk objek paling berbahaya
5	GPU atau `imgsz` lebih kecil	Deteksi 10 fps, bukan realtime

Langkah 1 dan 2 paling berdampak. Sisanya menambah kompleksitas sebelum dasar ukurannya cukup kuat.

---

10. Cara menjalankan

Aplikasi Windows (tanpa browser, tanpa terminal)

```
uv pip install --python .venv\Scripts\python.exe pywebview   (sekali saja)
double-click Start MotoTrack.bat
```

Browser

```
.venv\Scripts\python.exe -m mototrack.cli web
-> http://127.0.0.1:8770
```

CLI

```
.venv\Scripts\python.exe -m mototrack.cli analyze klip.mp4 --detector mixed
.venv\Scripts\python.exe -m mototrack.cli compare-detectors
.venv\Scripts\python.exe scripts\calibrate.py --detector mixed
.venv\Scripts\python.exe -m pytest tests
```

Kalau perbaikan kode tidak terlihat

Uvicorn tanpa `--reload` menyimpan kode yang ia mulai. Restart server **dan** hard-refresh browser (Ctrl+Shift+R). Keduanya perlu; salah satu saja menyisakan perilaku lama.

---

11. Lisensi & atribusi

Paket Ultralytics dan kedua checkpoint berlisensi **AGPL-3.0**. Proyek ini mewarisi kewajiban itu: kalau suatu saat dijadikan layanan jaringan, source harus tersedia.

Bobot tidak di-vendor ke repo; diunduh `scripts/fetch_models.py` ke `models/` yang di-gitignore. Footage dan gold set juga tidak masuk repo — bukan milik kita untuk diredistribusi.

Dataset fine-tune opsional: `visaitech/vehicle-mixed-traffic-detection`, CC-BY-4.0, 239 frame latih / 54 validasi.

12. Kesimpulan

Pertanyaan "bisa nyalip atau tidak" **tidak bisa dijawab model siap pakai**, karena model semacam itu tidak ada. Jadi jawabannya dibangun: deteksi dipinjam, keputusan dibuat sendiri, ambangnya diturunkan dari data.

Hasilnya berjalan end-to-end di CPU, di web dan sebagai aplikasi Windows. Angkanya ditulis apa adanya, termasuk yang buruk. Tujuh bug ditemukan lewat pengukuran, bukan lewat asumsi — dan tiap perbaikan punya tes yang terbukti menangkap regresi.

Nilai portofolio proyek ini bukan pada angka akurasinya, tapi pada caranya mengukur: menyatakan asumsi, mem-fit ambang ke data berlabel, melaporkan hasil negatif, dan membedakan "tampilan lebih halus" dari "model lebih pintar".